EMERGENCY MEDICINE AVAILABILITY NETWORK

MODULE 5

PAYMENT MANAGEMENT

Gateway-Independent Module Specification

Frontend	HTML, CSS, JavaScript, Bootstrap
Backend	Java, Spring Boot
Database	MySQL (integrate after inspecting existing schema)
Payment design	Payment-method selection independent of any single gateway

1. Module Overview

Module 5 manages payment choices and payment status for medicine orders in the Emergency Medicine Availability Network. It connects to the existing order flow (especially Module 4) and uses trusted prices and order information from the backend. The design must not depend on Razorpay or any single payment provider.

2. Objectives

- Provide a clear checkout page with payment options supported by the application configuration.
- Show medicine cost, applicable delivery charges, and a backend-validated total.
- Keep payment status and order/delivery status separate.
- Support online payment through a configured, secure payment provider or supported direct integration.
- Support Cash on Delivery only where the pharmacy permits it.
- Prevent duplicate payment processing and unauthorized access to payment records.

3. Payment Methods

Method	Implementation notes
UPI — GPay, PhonePe, Paytm	Offer through a supported UPI flow. App availability and the ability to complete payment depend on the selected integration and device.
UPI QR Code	Generate/display a legitimate payment QR through a supported merchant/payment integration. Verify the payment status before confirming an order.
Debit Card	Use a compliant payment provider or hosted checkout; never collect or store raw card number or CVV in this application.
Credit Card	Use a compliant payment provider or hosted checkout; never collect or store raw card number or CVV in this application.
Cash on Delivery (COD)	Show only when enabled by the pharmacy. Payment remains pending until cash collection is confirmed.

Important: displaying a payment option does not itself process or verify a payment. Real online transactions require a supported payment integration and secure backend verification.

4. Checkout Workflow

- User selects a valid existing medicine order.
- Backend retrieves current trusted prices, order details, eligible delivery fees, and order status.
- Checkout displays an itemized summary and only the methods enabled for that order/pharmacy.
- For online payments, the application initiates the configured integration and obtains an authoritative payment result.
- The backend verifies payment status before marking the payment as PAID.
- For COD, the status stays PENDING until payment collection is confirmed.
- The application displays the transaction reference and clear pending, success, failure, or cancellation states.

5. Integration with Existing Modules

Module	Responsibility / connection
Module 1 — User & Authentication	Authenticate users and restrict each user to their own orders and payment records.

Module	Responsibility / connection
Module 2 — Medicine & Inventory	Use trusted prices and stock validation; follow the existing inventory reservation/release rules.
Module 3 — Medicine Search & Availability	Pass selected medicine and pharmacy into the request/checkout flow; revalidate on the backend.
Module 4 — Emergency Request & Notification	Attach payment to an existing order and notify the user of relevant verified status changes.

6. Suggested Database Design

Inspect the existing schema first and reuse existing order/user tables. Add a payment table only if needed. Suggested fields:

Field	Purpose
id	Internal payment identifier
order_id	Reference to the existing order
amount, currency	Backend-calculated amount and currency
payment_method	UPI, UPI_QR, DEBIT_CARD, CREDIT_CARD, COD
provider_name	Optional integration/provider identifier; not hardcoded to one gateway
provider_order_ref	External order reference when supported
provider_payment_ref	External payment reference when supported
status	CREATED, PENDING, PAID, FAILED, CANCELLED, REFUNDED
created_at, updated_at	Audit timestamps

Never store raw card numbers, CVV, UPI PINs, or payment credentials. Keep any provider secrets in server-side environment variables.

7. Suggested Backend APIs

- POST /api/v1/payments — initiate payment for a valid order.
- POST /api/v1/payments/verify — verify the result using the configured integration.
- POST /api/v1/payments/callback — receive provider callback/webhook events where supported.
- GET /api/v1/payments/{paymentId} — read payment status for an authorized user.
- GET /api/v1/orders/{orderId}/payment — retrieve payment details for an authorized order.
- POST /api/v1/payments/{paymentId}/retry — retry an eligible payment safely.

Endpoint names are suggestions; align them with the existing project's API conventions and integration capabilities.

8. Security and Reliability

- Calculate the payable amount on the backend; do not trust totals sent by the browser.
- Use authoritative provider confirmation or a verifiable supported payment flow before marking payment PAID.
- Use idempotency and duplicate-event handling to prevent duplicate charges or confirmations.
- Authorize access to each order and payment record.
- Keep payment status separate from order and delivery status.
- Never simulate a successful real payment. Clearly label demo/test flows.
- Handle pending status, delayed callbacks, cancellations, and network failures safely.

9. Testing Checklist

- Successful test/demo payment with clearly identified mode
- Failed, pending, and cancelled payment
- Invalid or duplicated callback
- COD enabled and disabled by pharmacy
- Incorrect amount and invalid order status
- Unauthorized access to another user's payment
- Missing integration configuration or network failure
- Inventory changes between checkout and payment

10. Implementation Notes

This module is gateway-independent at the design level, but it does not mean every payment method can be processed without an integration. GPay, PhonePe, Paytm, UPI QR, and cards require a legitimate supported merchant payment flow to confirm real payments. COD can be handled by the application workflow if the pharmacy supports it. Select and configure the required integration before enabling real online payments.